FrogBard-512

Mathematical, Engineering, and Empirical Validation Report

Algorithm version 0.3-experimental · Document revision 0.3-eval1

A four-voice, 2048-bit permutation-based hash research prototype
512-bit digest · 1024-bit rate · 1024-bit capacity · 16-round permutation

Víctor Duarte Melo

19 June 2026

Libertas per Croack!

Security status

FrogBard-512 remains an unpublished experimental construction with no independent cryptanalysis. The evaluation reported here provides strong implementation-quality evidence and broad statistical smoke testing; it does not prove collision, preimage, second-preimage, indifferentiability, or structural cryptanalytic security. It is not recommended as a production replacement for standardized hashes.

Document status and revision scope

This revision preserves the complete mathematical and engineering specification of algorithm version 0.3-experimental and adds a detailed empirical validation report from the `insane` harness profile executed on 19 June 2026. No digest rule, constant, CroackBox, rotation, padding byte, tree encoding, or test vector is changed by this document revision.

The normative hierarchy remains:

1. `frogbard.c` and `frogbard_tree.c` for operation order and encodings;
2. `tools/gen_tables.py` for S-box, round-constant, and IV generation;
3. `frogbard_tables.h` for generated values;
4. fixed vectors in `TESTVECTORS.txt` and `TREEVECTORS.txt`.

The original 24-page v0.3 specification supplied for this revision is reproduced as Part I, excluding its superseded cover page. Part II adds the new test methodology, corrected result accounting, charts, limitations, and recommended next work. The uploaded source specification is the basis for the normative material in Part I.[1]

Key finding

The completed run lasted **16,842 seconds** (4 h 40 min 42 s), generated approximately **17 GiB** of artifacts, executed one million randomized API cases, ran two one-hour fuzzers, and tested two independent 8 GiB output streams. No crash, memory error, detected race, leak, backend mismatch, fixed-vector mismatch, or statistical test failure was observed in the completed tests.

Abstract

FrogBard-512 maps arbitrary finite byte strings to 512-bit digests using a custom 2048-bit permutation organized as four 512-bit voices. Each round combines public constants, four affine-equivalent AES-derived CroackBoxes, ARX quarter-rounds, a parity-dependent cross-voice braid, and fixed lane permutations. Sequential hashing uses a 1024-bit rate and 1024-bit capacity; a separately domain-separated tree mode uses 1 MiB leaves, multithreading, and an AVX2 four-way backend.

This revision reports the first large integrated validation campaign. The `insane` profile recorded 250 raw passes, nine raw failures, and six warnings. Re-analysis shows that two raw failures were successful PractRand runs misclassified by shell `pipefail`/SIGPIPE behavior, six were user-interrupted performance or resource-limit tests, and one was a Cppcheck warning requiring code review. Completed evidence includes: 1,481,187 libFuzzer executions with zero crashes; one hour of AFL++ with zero crashes and zero timeouts; 1,000,000 randomized API cases; AddressSanitizer, LeakSanitizer, UndefinedBehaviorSanitizer, ThreadSanitizer, Memcheck, Helgrind, and DRD checks; reproducible constant and binary generation; 16,384 inverse-permutation samples per round for rounds 1–16; 8 GiB counter and 8 GiB one-bit differential streams with no PractRand anomalies; and ideal-like full-hash avalanche statistics over one million cases. These results materially strengthen implementation confidence but do not establish cryptographic security.

[1] Source document: *FrogBard-512 v0.3 Mathematical and Engineering Specification*, 24 pages, dated 19 June 2026.

Contents

Executive result dashboard

Run profile	`insane`; 32 jobs; total elapsed 4 h 40 min 42 s
Platform	Ubuntu 26.04 LTS; Linux 7.0.0-22; AMD Ryzen 9 5950X; 128 GiB-class RAM
Toolchain	Clang/LLD 21.1.8; GCC 15.2; Valgrind 3.26; AFL++ 4.33c; PractRand 0.96
Raw harness summary	250 PASS, 9 FAIL, 6 WARN
Corrected interpretation	252 completed without observed anomaly; 1 static-analysis warning; 6 interrupted tests
Dynamic testing	1,481,187 libFuzzer executions; 1 h AFL++; 1,000,000 randomized API cases
Random-stream volume	8 GiB counter stream + 8 GiB one-bit differential stream
PractRand	270 results per stream; no anomalies in either stream
Dieharder	224 PASSED, 4 WEAK, 0 FAILED across both batteries
Full-hash avalanche	mean 256.002533; std. dev. 11.310838; ideal std. dev. $\sqrt{128}$ = 11.313708
LLVM coverage	80.50% lines, 66.10% branches, 96.30% functions overall
Memory/race tools	zero Memcheck errors; no leaks; zero Helgrind/DRD errors; sanitizer runs passed
Security conclusion	strong implementation evidence; no production-security claim

Table 1: High-level outcome of the 19 June 2026 evaluation.

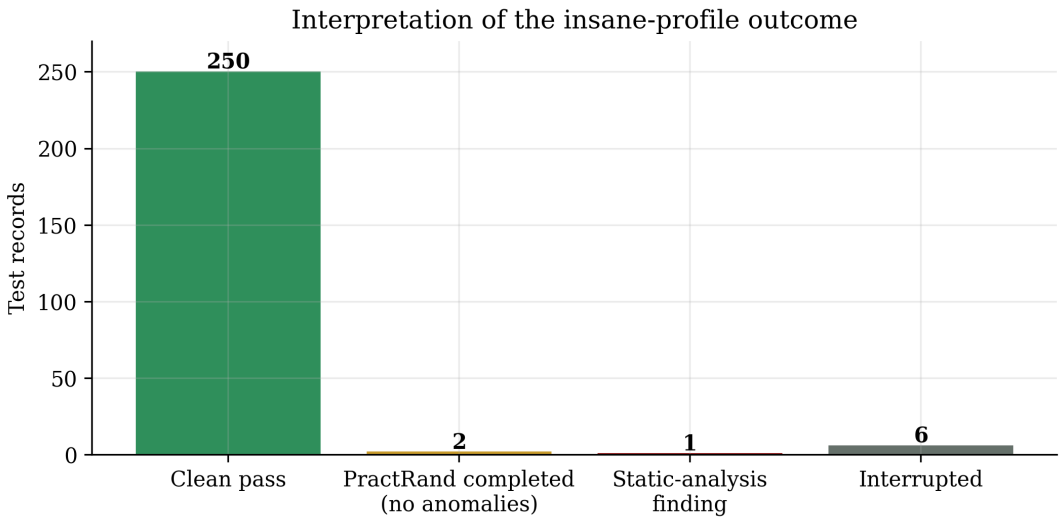


Figure 1: Corrected classification of the raw harness result. The two PractRand records completed their requested 8 GiB analyses with no anomalies but returned 141 because the upstream generator received SIGPIPE after PractRand closed the pipe.

Part I

Normative Mathematical and Engineering Specification

Reproduced from the v0.3-experimental source specification

Interpretation note

The following pages are reproduced unchanged to preserve normative accuracy. Their internal table of contents and printed page numbers refer to the original standalone specification. Part II begins after the reproduced specification.

Document status and normative hierarchy

This document expands the repository specification into a self-contained mathematical and engineering description. In case of discrepancy, the v0.3 source artifacts are normative in the following order:

1. `frogbard.c` and `frogbard_tree.c` for operation order and encodings;
2. `tools/gen_tables.py` for S-box, round-constant, and IV generation;
3. `frogbard_tables.h` for the generated values;
4. the fixed vectors in `TESTVECTORS.txt` and `TREEVECTORS.txt`.

The source archive used for this document has SHA-256 digest:

`2870859f42e6ec893ba3a53c53a9302edc2193e3240c41fbae20c3fb06445d43`.

Abstract

FrogBard-512 maps arbitrary finite byte strings to 512-bit digests. Its sequential mode is a sponge-like construction over a custom 2048-bit permutation. The state consists of four 512-bit "voices," each represented by eight 64-bit words. Every round combines public round constants, one of four affine-equivalent AES S-boxes, ARX quarter-rounds, a parity-dependent cross-voice braid, and a fixed lane permutation. The sequential rate is 1024 bits and the capacity is 1024 bits. A separately domain-separated tree mode uses 1 MiB leaves, a four-way AVX2 backend, and explicit root binding. This document defines the byte conventions, deterministic constant generator, round function, inverse permutation, absorption, padding, finalization, tree encoding, implementation profiles, conformance vectors, and current experimental evidence. It also identifies the limits of that evidence and the cryptanalytic work still required.

Contents

List of Figures

List of Tables

1 Scope, goals, and non-goals

1.1 Functional definition

The sequential hash is a function

$$H_{\mathrm{FB}} : \{0,1\}^* \longrightarrow \{0,1\}^{512}.$$

Inputs are processed as byte strings. The implementation supplies incremental, one-shot, four-way equal-length, and regular-file tree APIs.

1.2 Design goals

The v0.3 design targets the following engineering properties:

- a wide 2048-bit internal state with a 1024-bit hidden capacity;
- rapid diffusion through byte substitution, ARX mixing, cross-voice coupling, and lane movement;
- a small hot working set suitable for L1 cache;
- a portable C11 scalar implementation and a constant-time bitsliced profile;
- independent-message SIMD through an AVX2 four-way backend;
- streaming operation without dynamic allocation in the sequential core;
- a deterministic and publicly reproducible constant-generation procedure;
- an exact inverse for reduced-round permutation testing;
- explicit domain separation for tree leaves, parents, and the bound root.

1.3 Non-goals

Version 0.3 is not a password hashing function, KDF, MAC, signature primitive, or standardized XOF. It does not claim resistance beyond what follows from independent cryptanalysis. The public phrases used to derive constants are not keys and add no secrecy.

2 Notation and conventions

2.1 Word ring and bit operations

Let

$$\mathbb{W} = \mathbb{Z}/2^{64}\mathbb{Z}.$$

All additions and subtractions on state words are in $\mathbb{W}$. We write

$$x \boxplus y = (x + y) \bmod 2^{64}, \qquad x \boxminus y = (x - y) \bmod 2^{64}.$$

Bitwise exclusive OR is $\oplus$. Left rotation of a 64-bit word by n positions is

$$\mathrm{ROTL}_n(x) = ((x \ll n) \vee (x \gg (64 - n))) \bmod 2^{64},$$

with n reduced modulo 64. Its inverse is ROTR_n.

2.2 Byte encoding

External 64-bit words are decoded and encoded in little-endian order:

$$\mathrm{LE64}(b_0, \ldots, b_7) = \sum_{i=0}^{7} b_i 2^{8i}.$$

The digest is the little-endian serialization of eight state words and is exactly 64 bytes.

2.3 State indexing

The state is

$$V = (V_0, V_1, V_2, V_3) \in \mathbb{W}^{4\times 8},$$

where

$$V_v = (v_{v,0}, v_{v,1}, \ldots, v_{v,7}).$$

The total state size is

$$4 \times 8 \times 64 = 2048 \text{ bits}.$$

Voices V_0 and V_1 are the 1024-bit rate; V_2 and V_3 are the 1024-bit capacity.

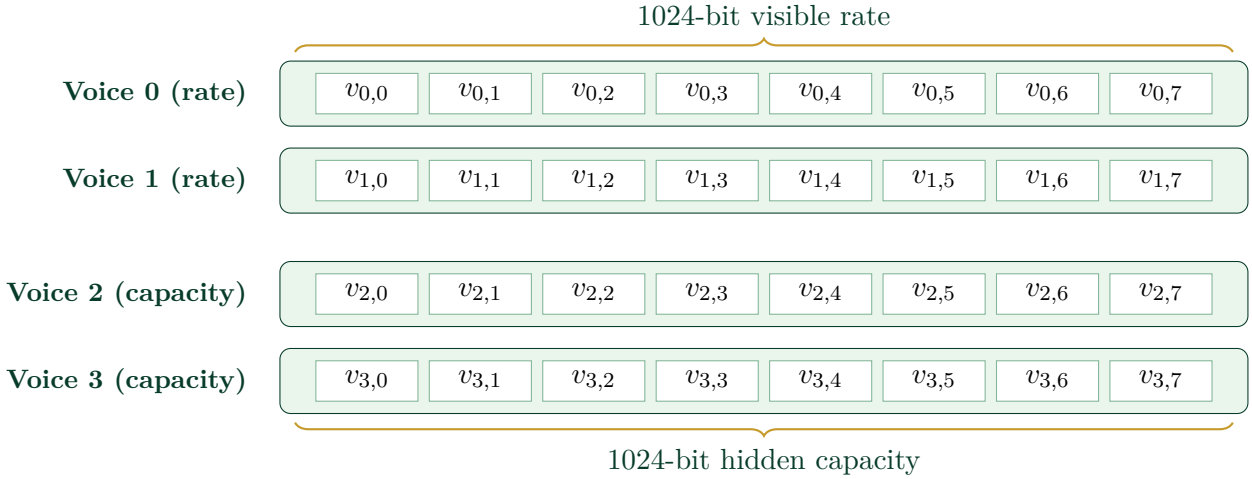


Figure 1: The four-voice state organization. Each lane is one 64-bit word.

Table 1: Core parameters.

Parameter	Value
Digest size	512 bits (64 bytes)
Internal state	2048 bits (32 words)
Rate	1024 bits (voices 0 and 1)
Capacity	1024 bits (voices 2 and 3)
Message block	128 bytes
Permutation rounds	16
Final permutation calls	2 additional calls
Sequential byte order	little endian
Tree leaf payload	at most 1,048,576 bytes

3 Public derivation phrases and reproducible constants

3.1 Exact phrase bytes

The four phrases are exact ASCII byte strings with no trailing newline:

1. `Libertas per Croack!`
2. `Presunto`
3. `Floquinha`

4. `In Frog we Trust!`

They provide reproducible, human-readable seeds. They are public and must never be interpreted as secret key material.

3.2 Project-defined 64-bit derivation mixer

The generator uses a simple deterministic mixer, not a cryptographic hash. Define constants

$$F_0 = 1469598103934665603, \qquad F_p = 1099511628211.$$

The first value is intentionally documented exactly as used by v0.3; implementations must not silently substitute another FNV offset basis.

For domain bytes D, phrase bytes P, and unsigned 64-bit counter c, define $\mathrm{Mix64}(D, P, c)$ by the following algorithm, with every multiplication reduced modulo 2^{64}:

Listing 1: Normative project-defined FNV-style mixer.

```
h = 1469598103934665603
for b in D:
    h = (h XOR b) * 1099511628211 mod 2^64
h = (h XOR 0x00) * 1099511628211 mod 2^64
for b in P:
    h = (h XOR b) * 1099511628211 mod 2^64
for i = 0..7:
    b = (c >> (8*i)) AND 0xff
    h = (h XOR b) * 1099511628211 mod 2^64
return h
```

The explicit zero byte separates the domain from the phrase. The counter is appended as eight little-endian bytes.

3.3 SplitMix64 expansion

A state $s \in \mathbb{W}$ is advanced and expanded by

$$\begin{aligned}
s' &= s \boxplus \texttt{9E3779B97F4A7C15}_{16}, \\
z_0 &= s', \\
z_1 &= (z_0 \oplus (z_0 \gg 30)) \cdot \texttt{BF58476D1CE4E5B9}_{16}, \\
z_2 &= (z_1 \oplus (z_1 \gg 27)) \cdot \texttt{94D049BB133111EB}_{16}, \\
z_3 &= z_2 \oplus (z_2 \gg 31).
\end{aligned}$$

The generator returns (s', z_3). All multiplications are modulo 2^{64}.

3.4 Bit permutations

For an eight-element permutation $p = (p_0, \ldots, p_7)$, define

$$\mathrm{BP}_p(x) = \sum_{i=0}^{7} \left(\left(\frac{x}{2^{p_i}} \right) \bmod 2 \right) 2^i.$$

Thus output bit i receives input bit p_i. Candidate permutations are generated by Fisher-Yates shuffling of $(0, 1, \ldots, 7)$, consuming one SplitMix64 output per swap and selecting $j = z \bmod (i+1)$.

4 CroackBoxes

4.1 Definition

Let $S_{\mathrm{AES}} : \mathbb{F}_2^8 \to \mathbb{F}_2^8$ be the standard AES S-box. For phrase index $j \in \{0, 1, 2, 3\}$ and candidate counter c, the generator derives input and output bit permutations $p_j^{\mathrm{in}}, p_j^{\mathrm{out}}$ and bytes a_j, b_j. The candidate CroackBox is

$$S_j(x) = \mathrm{BP}_{p_j^{\mathrm{out}}}\left(S_{\mathrm{AES}}\left(\mathrm{BP}_{p_j^{\mathrm{in}}}(x \oplus a_j)\right)\right) \oplus b_j.$$

The candidate is rejected if any x satisfies either

$$S_j(x) = x \qquad \text{or} \qquad S_j(x) = x \oplus \mathtt{FF}_{16}.$$

The counter is incremented until both classes of fixed points are absent.

4.2 Exact affine metadata

Table 2: Derived CroackBox metadata for v0.3.

j	Phrase	p_j^{in}	p_j^{out}	a_j	b_j
0	`Libertas per Croack!`	$[3,6,5,7,1,0,4,2]$	$[7,6,0,3,4,5,2,1]$	`A7`	`71`
1	`Presunto`	$[3,7,6,0,2,4,1,5]$	$[3,0,2,5,6,4,1,7]$	`72`	`43`
2	`Floquinha`	$[5,3,2,4,7,6,1,0]$	$[6,7,3,4,1,5,0,2]$	`8F`	`9E`
3	`In Frog we Trust!`	$[5,7,4,2,3,1,6,0]$	$[1,3,6,2,7,0,4,5]$	`8A`	`76`

The accepted candidate counters are respectively $11, 6, 10, 18$.

4.3 Preserved properties

Input and output bit permutations are invertible linear maps over $\mathbb{F}_2^8$, and XOR by a constant is affine. Therefore every S_j is affine-equivalent to the AES S-box. In particular:

- each S_j is bijective;
- differential uniformity remains 4;
- nonlinearity and algebraic degree are preserved under affine equivalence;
- each inverse is computable by reversing the affine maps and applying S_{AES}^{-1}.

These properties concern the isolated byte maps. They do not prove security of the full 2048-bit permutation.

4.4 Word-level application

For $x \in \mathbb{W}$, with byte decomposition $x = \sum_{k=0}^{7} x_k 2^{8k}$,

$$\widehat{S}_j(x) = \sum_{k=0}^{7} S_j(x_k) 2^{8k}.$$

The default scalar profile evaluates this with a 256-byte lookup table. The constant-time scalar profile evaluates the same map using a bitsliced Boyar-Peralta AES S-box circuit plus the derived affine transformations.

5 Round constants and initial state

5.1 Round-constant seed

Let

$$\sigma_0 = \texttt{46524F4742415244}_{16},$$

whose hexadecimal bytes spell `FROGBARD`. For phrase index $p = 0, 1, 2, 3$:

$$\begin{aligned} \sigma &\leftarrow \sigma \oplus \mathrm{Mix64}(\texttt{FrogBard-512/ROUND-CONSTANTS/v1}, P_p, p), \\ (\sigma, z) &\leftarrow \mathrm{SplitMix64}(\sigma), \\ \sigma &\leftarrow z. \end{aligned}$$

5.2 Round-constant words

For round $r \in [0, 15]$, voice $v \in [0, 3]$, and lane $\ell \in [0, 7]$, consume one SplitMix64 output x and define

$$\mathrm{RC}_{r,v,\ell} = x \oplus (\texttt{9E3779B97F4A7C15}_{16}(r+1)) \oplus \mathrm{ROTL}_{\ell+1}\left(\texttt{D1B54A32D192ED03}_{16}(v+1)\right),$$

with products and rotations in $\mathbb{W}$. There are $16 \times 4 \times 8 = 512$ round-constant words.

5.3 Initial state

For each voice v, initialize

$$s_v = \mathrm{Mix64}(\texttt{FrogBard-512/IV/v1}, P_v, v).$$

For each lane ℓ, consume $(s_v, z) = \mathrm{SplitMix64}(s_v)$ and define

$$\mathrm{IV}_{v,\ell} = z \oplus \mathrm{RC}_{0,v,\ell} \oplus \mathrm{ROTL}_{7\ell+v}(\sigma_0).$$

The generated table header has SHA-256 digest

`7a4d1b7bdd9aad454ce7bb513d9126608695afea2db33b5ab26a900608dabd9`.

This checksum is a convenient conformance aid; the mathematical generator remains the authoritative definition.

6 The ARX quarter-round

6.1 Forward map

For $(a, b, c, d) \in \mathbb{W}^4$ and rotation tuple (r_0, r_1, r_2, r_3), define Q_{r_0,r_1,r_2,r_3} by

$$\begin{aligned} a &\leftarrow a \boxplus b, \\ d &\leftarrow \mathrm{ROTL}_{r_0}(d \oplus a), \\ c &\leftarrow c \boxplus d, \\ b &\leftarrow \mathrm{ROTL}_{r_1}(b \oplus c), \\ a &\leftarrow a \boxplus b, \\ d &\leftarrow \mathrm{ROTL}_{r_2}(d \oplus a), \\ c &\leftarrow c \boxplus d, \\ b &\leftarrow \mathrm{ROTL}_{r_3}(b \oplus c). \end{aligned}$$

This is a 256-bit bijection composed only of modular addition, XOR, and rotation.

6.2 Exact inverse

The inverse applies the primitive inverses in reverse order:

$$\begin{aligned} b &\leftarrow \mathrm{ROTR}_{r_3}(b) \oplus c, & c &\leftarrow c \boxminus d,\\ d &\leftarrow \mathrm{ROTR}_{r_2}(d) \oplus a, & a &\leftarrow a \boxminus b,\\ b &\leftarrow \mathrm{ROTR}_{r_1}(b) \oplus c, & c &\leftarrow c \boxminus d,\\ d &\leftarrow \mathrm{ROTR}_{r_0}(d) \oplus a, & a &\leftarrow a \boxminus b. \end{aligned}$$

Hence $Q^{-1}(Q(a,b,c,d)) = (a,b,c,d)$ for every input.

7 Within-voice mixing

7.1 Rotation families

Let the four base tuples be

$$\begin{aligned} R_0 &= (17, 29, 41, 53),\\ R_1 &= (23, 31, 47, 59),\\ R_2 &= (11, 37, 43, 57),\\ R_3 &= (19, 27, 45, 61). \end{aligned}$$

Voice v in round r selects

$$R_{(v+r) \bmod 4} = (\rho_0, \rho_1, \rho_2, \rho_3).$$

7.2 Four quarter-round schedule

For a voice $X = (x_0, \ldots, x_7)$, the mixer $M_{v,r}$ executes, in this exact order:

1. $Q_{\rho_0,\rho_1,\rho_2,\rho_3}(x_0, x_1, x_2, x_3)$;
2. $Q_{\rho_1,\rho_2,\rho_3,\rho_0}(x_4, x_5, x_6, x_7)$;
3. $Q_{\rho_2,\rho_3,\rho_0,\rho_1}(x_0, x_5, x_2, x_7)$;
4. $Q_{\rho_3,\rho_0,\rho_1,\rho_2}(x_4, x_1, x_6, x_3)$.

The first two quarter-rounds mix disjoint lane quartets. The latter two create cross-links between those quartets. The inverse executes the four inverse quarter-rounds in reverse schedule order.

8 Cross-voice braid

8.1 Rotation vectors

Define

$$\begin{aligned} A &= (7, 13, 19, 29, 37, 43, 53, 61),\\ B &= (11, 17, 23, 31, 41, 47, 55, 59). \end{aligned}$$

All lane indices are reduced modulo 8. The loop index i advances from 0 through 7. Operations are sequential: later operations can read words modified by earlier operations, including prior values of i.

8.2 Even rounds

For even r, and for each $i = 0, \ldots, 7$ in ascending order:

$$\begin{aligned} v_{0,i} &\leftarrow v_{0,i} \boxplus \mathrm{ROTL}_{A_i}(v_{1,i+1}), \\ v_{2,i} &\leftarrow v_{2,i} \oplus \mathrm{ROTL}_{B_i}(v_{0,i+3}), \\ v_{3,i} &\leftarrow v_{3,i} \boxplus \mathrm{ROTL}_{A_{i+2}}(v_{2,i+5}), \\ v_{1,i} &\leftarrow v_{1,i} \oplus \mathrm{ROTL}_{B_{i+4}}(v_{3,i+7}). \end{aligned}$$

8.3 Odd rounds

For odd r:

$$\begin{aligned} v_{3,i} &\leftarrow v_{3,i} \boxplus \mathrm{ROTL}_{B_i}(v_{2,i+2}), \\ v_{1,i} &\leftarrow v_{1,i} \oplus \mathrm{ROTL}_{A_i}(v_{3,i+4}), \\ v_{0,i} &\leftarrow v_{0,i} \boxplus \mathrm{ROTL}_{B_{i+3}}(v_{1,i+6}), \\ v_{2,i} &\leftarrow v_{2,i} \oplus \mathrm{ROTL}_{A_{i+5}}(v_{0,i+1}). \end{aligned}$$

8.4 Invertibility

The braid is inverted by iterating i from 7 down to 0 and undoing the four assignments in reverse order, replacing modular addition by subtraction while retaining XOR as its own inverse. The descending index order is required because the forward map has loop-carried dependencies.

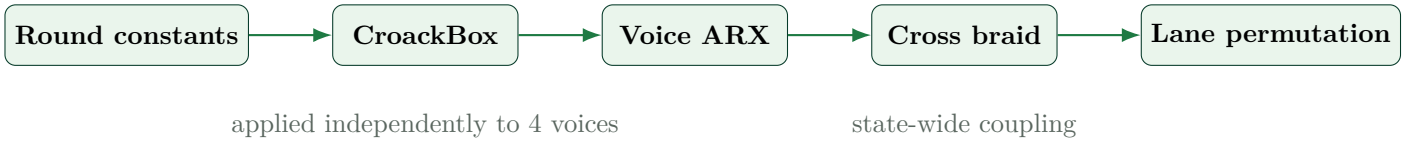


Figure 2: One FrogBard permutation round.

9 Lane permutations

For each voice, the output lane j receives the input lane at the listed index. That is,

$$V'_v[j] = V_v[\pi_{r,v}[j]].$$

Table 3: Parity-dependent lane permutations.

Parity	Voice	$\pi_{r,v}$
Even	0	$[0, 3, 6, 1, 4, 7, 2, 5]$
Even	1	$[5, 0, 3, 6, 1, 4, 7, 2]$
Even	2	$[2, 5, 0, 3, 6, 1, 4, 7]$
Even	3	$[7, 2, 5, 0, 3, 6, 1, 4]$
Odd	0	$[1, 6, 3, 0, 5, 2, 7, 4]$
Odd	1	$[4, 1, 6, 3, 0, 5, 2, 7]$
Odd	2	$[7, 4, 1, 6, 3, 0, 5, 2]$
Odd	3	$[2, 7, 4, 1, 6, 3, 0, 5]$

Every row is a permutation of $\{0, \ldots, 7\}$. The inverse map places each output word back into its source position.

10 Full permutation

10.1 Round definition

Let K_r denote round-constant XOR, S_r the four voice-dependent CroackBoxes, M_r the four within-voice ARX mixers, B_r the cross-voice braid, and Π_r the lane permutations. A round is

$$\mathcal{R}_r = \Pi_r \circ B_r \circ M_r \circ S_r \circ K_r.$$

The full permutation is

$$P = \mathcal{R}_{15} \circ \mathcal{R}_{14} \circ \cdots \circ \mathcal{R}_0.$$

10.2 Bijection theorem

Permutation bijection

Each component of $\mathcal{R}_r$ is bijective: XOR with a constant is self-inverse; every CroackBox is bijective; the ARX mixer is a composition of bijective quarter-rounds; the braid has an explicit inverse; and the lane movement is a permutation. Therefore every $\mathcal{R}_r$ and the 16-round composition P are bijections on $\{0,1\}^{2048}$.

The research API exposes P_t and P_t^{-1} for any prefix of $t \in [1, 16]$ rounds. This enables exact inversion tests without implying that the final hash is invertible.

10.3 Why a reversible permutation can produce a one-way hash

The compression mechanism does not publish the complete final state and does not expose intermediate capacity words. The hash maps an unbounded input domain to 512 bits:

$$H_{\mathrm{FB}} : \{0,1\}^* \to \{0,1\}^{512}.$$

By the pigeonhole principle, infinitely many messages share each output on average. Reversibility of P is a structural requirement of the sponge-like core, not an inverse for the complete hash mapping.

11 Sequential hashing

11.1 Initialization

A context is zeroed, the generated IV is copied into the state, and one full permutation is applied:

$$V^{(0)} = P(\mathrm{IV}).$$

The byte counter is a 128-bit unsigned count represented by $(L_{\mathrm{hi}}, L_{\mathrm{lo}})$.

11.2 Block absorption

A 128-byte block M_j is decoded into sixteen words $(m_0, \ldots, m_{15})$. The rate injection is

$$\begin{aligned} v_{0,i} &\leftarrow v_{0,i} \oplus m_i, \\ v_{1,i} &\leftarrow v_{1,i} \oplus m_{i+8}, \qquad i = 0, \ldots, 7. \end{aligned}$$

Then

$$V \leftarrow P(V).$$

Capacity voices 2 and 3 are not directly XORed with message words.

11.3 Streaming semantics

The API buffers fewer than 128 bytes. A full buffer is absorbed immediately. Large input calls absorb complete blocks directly from the caller's memory. The total byte counter is updated independently of buffering, so segmentation of the same byte string into update calls cannot change the digest.

11.4 Padding

Let $\ell \in [0, 127]$ be the number of buffered bytes. A zero-filled 128-byte final block is constructed, the remaining bytes are copied, and

$$B[\ell] \leftarrow B[\ell] \oplus \texttt{0B}_{16}, \qquad B[127] \leftarrow B[127] \oplus \texttt{80}_{16}.$$

If $\ell = 127$, the last byte receives both markers and becomes $\texttt{8B}_{16}$. If the message length is a multiple of 128, $\ell = 0$ and an additional padding block is still absorbed. This makes the encoding injective with respect to complete-block boundaries.

11.5 Length and final-domain injection

After the padded block has been absorbed and permuted:

$$\begin{aligned} v_{2,6} &\leftarrow v_{2,6} \oplus L_{\mathrm{lo}}, \\ v_{2,7} &\leftarrow v_{2,7} \oplus L_{\mathrm{hi}}, \\ v_{3,6} &\leftarrow v_{3,6} \oplus \texttt{496E2046726F6720}_{16}, \\ v_{3,7} &\leftarrow v_{3,7} \oplus \texttt{7765205472757374}_{16}. \end{aligned}$$

The latter two hexadecimal literals visually encode `In Frog` and `we Trust` in conventional hexadecimal display order. They are injected as 64-bit integer values; the state itself follows the defined word semantics rather than a textual byte interpretation.

11.6 Two final permutations

Two marker-separated full permutations follow:

$$\begin{aligned} v_{3,0} &\leftarrow v_{3,0} \oplus \texttt{21}_{16}, & V &\leftarrow P(V), \\ v_{3,1} &\leftarrow v_{3,1} \oplus \texttt{2100}_{16}, & V &\leftarrow P(V). \end{aligned}$$

The markers occupy distinct voice/lane/byte positions, preventing the two final calls from being identical iterations of the same unmodified state.

11.7 Extraction

The digest is voice 0 serialized in little-endian order:

$$H_{\mathrm{FB}}(M) = \mathrm{LE64}(v_{0,0}) \,\|\, \mathrm{LE64}(v_{0,1}) \,\|\, \cdots \,\|\, \mathrm{LE64}(v_{0,7}).$$

Listing 2: Sequential hashing pseudocode.

```
function FrogBard512(message):
    V = IV
    V = P(V)

    for every complete 128-byte block M:
        V[0][0..7] XOR= LE64(M[0..63])
        V[1][0..7] XOR= LE64(M[64..127])
```

```
        V = P(V)

    B = zero[128]
    copy remaining bytes into B
    B[remaining_length] XOR= 0x0b
    B[127] XOR= 0x80
    absorb B and apply P

    V[2][6] XOR= total_length_low64
    V[2][7] XOR= total_length_high64
    V[3][6] XOR= 0x496e2046726f6720
    V[3][7] XOR= 0x7765205472757374

    V[3][0] XOR= 0x21
    V = P(V)
    V[3][1] XOR= 0x2100
    V = P(V)

    return little_endian(V[0][0..7])
```

12 Tree hashing

12.1 Purpose and separation

Tree mode is intended for large regular files and exploits multiple threads plus four-way SIMD. It is a distinct hash domain:

$$H_{\text{tree}}(M) \neq H_{\text{FB}}(M)$$

in general, even when the entire file fits in one leaf.

12.2 Leaf encoding

The leaf payload size is

$$C = 2^{20} = 1{,}048{,}576 \text{ bytes.}$$

For chunk C_i, leaf index i, and chunk length $|C_i|$, define

$$L_i = H_{\text{FB}}\left(D_L \parallel \text{LE64}(i) \parallel \text{LE64}(|C_i|) \parallel C_i\right),$$

where D_L is the exact 16-byte string

$$\texttt{FrogBardTreeL1} \parallel \texttt{00} \parallel \texttt{00}.$$

An empty file has one zero-length leaf.

12.3 Parent encoding

A parent message is exactly 160 bytes:

$$D_P \parallel \text{LE64}(\ell) \parallel \text{LE64}(k) \parallel L \parallel R,$$

where $D_P = \texttt{FrogBardTreeP1} \parallel \texttt{00} \parallel \texttt{00}$, ℓ is the zero-based reduction level, and $k \in \{1, 2\}$ is the child count. L and R are 64-byte digests. When $k = 1$, R is 64 zero bytes. The parent digest is the sequential FrogBard hash of this structure.

12.4 Root binding

After reduction to one internal root digest R_{int}, the final 112-byte root message is

$$D_R \parallel \text{LE64}(F) \parallel \text{LE64}(C) \parallel \text{LE64}(N) \parallel \text{LE64}(h) \parallel R_{\text{int}},$$

where:

- $D_R = \texttt{FrogBardTreeR1} \parallel \texttt{00} \parallel \texttt{00}$;
- F is file size in bytes;
- $C = 2^{20}$ is the chunk size;
- N is the leaf count;
- h is the number of parent-reduction levels.

The final result is

$$H_{\text{tree}}(M) = H_{\text{FB}}(\text{root message}).$$

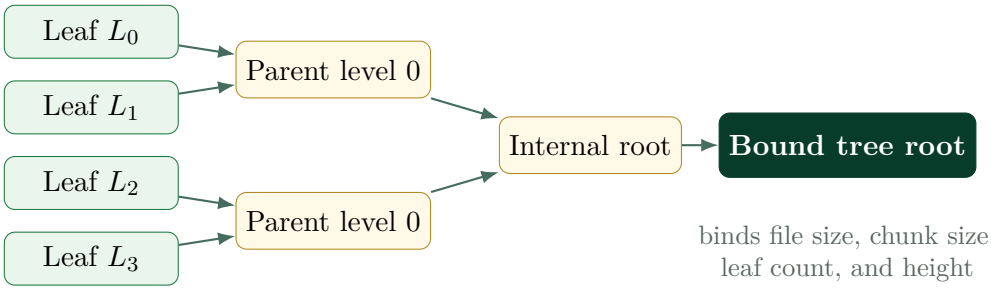


Figure 3: Domain-separated binary tree mode.

12.5 Parallel execution model

Workers claim groups of up to four leaves through an atomic counter and read the file with `pread()`. Four equal-length messages are processed by the AVX2 backend. The final partial group falls back to scalar calls when lengths differ. Parent nodes are similarly evaluated in four-way batches when possible. Thread count is capped by the number of four-leaf groups and by an implementation-defined maximum.

13 Implementation profiles

13.1 Default scalar table profile

The forward path uses four 256-byte S-box tables, totaling 1 KiB. State words and the 128-byte input buffer are aligned to 64 bytes. The forward hot set consists principally of the 256-byte state, 1 KiB S-box set, and 4 KiB round-constant array, all small enough for typical L1 data caches.

Table lookup addresses depend on message-derived state bytes. This is normally acceptable for an unkeyed public hash, but the profile is not constant-time with respect to its input.

13.2 Constant-time scalar profile

The `constant-time` build uses an orthogonal bitslice transform, the Boyar-Peralta AES S-box Boolean circuit, and the derived affine metadata. It produces exactly the same digest as the table profile while avoiding data-dependent S-box lookup addresses. The source includes a full all-byte equivalence test for each of the four boxes.

13.3 AVX2 four-way profile

The SIMD backend hashes four independent equal-length messages. Each 256-bit vector carries the corresponding 64-bit word of four separate states:

$$X_{v,\ell} = \left(v_{v,\ell}^{(0)}, v_{v,\ell}^{(1)}, v_{v,\ell}^{(2)}, v_{v,\ell}^{(3)}\right).$$

This organization preserves the scalar algorithm exactly and does not reinterpret four vector lanes as parts of one state. The backend is therefore naturally suited to tree leaves and parent nodes.

13.4 Clang and LLD build

The native build uses C11, Clang, ThinLTO, and LLD. The principal optimization flags are:

Listing 3: Native build profile.

```
clang -std=c11 -O3 -march=native -mtune=native \
      -flto=thin -ffunction-sections -fdata-sections \
      ... -fuse-ld=lld -Wl,--gc-sections
```

The repository also defines portable, strict-warning, debug, UBSan, ASan, hardened, static-library, analysis, fuzzing, and assembly-output targets.

14 Conformance requirements

An implementation is conformant to v0.3 sequential hashing only if all of the following hold:

1. the external byte order is little endian exactly as specified;
2. state arithmetic wraps modulo 2^{64};
3. the four phrases contain no trailing newline or alternate Unicode encoding;
4. the project-defined 64-bit mixer uses the exact offset basis written in Section 4;
5. S-box candidate rejection, counters, permutations, and XOR bytes match Table 2;
6. every round operation and every loop ordering is preserved;
7. padding always adds a final block and uses the `0B/80` markers;
8. the 128-bit byte length and final-domain constants are injected in the specified words;
9. two final permutations and their distinct markers are executed;
10. output is exactly voice 0 in little-endian order.

Tree conformance additionally requires the exact domains, fixed structured lengths, one-leaf empty-file rule, zero right child for unary parents, and root binding fields.

15 Fixed test vectors

15.1 Sequential vectors

Table 4: Sequential FrogBard-512 v0.3 vectors.

Input	Digest (hexadecimal)
Empty	`5a6e1f5691b551d2415093151447d619f719ad920bb8061f0f88a586e7a9cf60b2086bc65d57 7dbabccedd0a47c0adaeb2523f954ac97afbdb1a01acaa3dd961`
`abc`	`d73421ee1419c7ba0a5da91d9e728c5b4383e26337dec3c377ca53f656b556b11bc924b86771 63e88187a31481f553e2c1c4abc48fe213ce11a1208d71474113`

Input	Digest (hexadecimal)
`Libertas per Croack!`	`f8241e704c480e37481d703ae3f4525f8005785f441e5e3102667bb006517627ac0cdfb0cc35160f173c494e66cd85b041c97aab5016a405374d26280d2673ec`
`Presunto`	`67a82a70ed0b7bd1ea2922a191edb92060e9b716b67b4609588eef10665c6d7bfeb52335e9119b1e71c230f0e2c95cfd2dafbbd4fe1c4b0c7fe8ed76b051c941`
`Floquinha`	`324ebac7abf052e9eba5a6296ed51b2e34a47de20bcf747337c76fe8edcacaa1aa9541132159e61461aa0ee3ab63f527e78bb6f3e5c5219e6219dff8c9e21f8b`
`In Frog we Trust!`	`0b9247bfda60e4044b52ba8920e1cede7ce09777f212a84846cbda1873885e4fecab572659d19c402d2b6e134ba58e6d6d5bc02f4b31de0a3a6dd4d2653b3c4a`

15.2 Tree vectors

Table 5: Tree-mode vectors.

File bytes	Tree digest (hexadecimal)
Empty	`8ea132bfaa2f917c7cf0d90eb7912ce7035ad2f568bc9ac8f9af87cc63d86d05e6cb40d8f8469b47606249a8b789ede703f8217933e299ffe838d31d9b3f4be9`
`abc`	`0886b02955bf455af0d7371a8e1bb3bb49afa13a6a8c9241844062da178c55bf28942fc0eb1511a5b0e6e62acaea904be0534e9a905a5b277c8874d22d82e5b2`

16 Generic security targets and their limits

16.1 Idealized generic bounds

For a sponge-like construction with capacity $c = 1024$ and output length $n = 512$, an ideal-permutation heuristic suggests generic targets of approximately

$$\begin{aligned} \text{collision resistance} &\approx 2^{\min(n/2,c/2)} = 2^{256}, \\ \text{preimage resistance} &\approx 2^{\min(n,c)} = 2^{512}, \\ \text{second-preimage resistance} &\approx 2^{\min(n,c)} = 2^{512}. \end{aligned}$$

These are design targets under an idealized model, not proven security levels for the custom permutation.

16.2 Why state size is not a proof

A large capacity cannot compensate for a structural weakness in P. Reduced-round differential trails, invariant subspaces, rotational relations, algebraic shortcuts, meet-in-the-middle decompositions, or exploitable interactions between the S-box and ARX layers could lower the effective complexity. Security must therefore be argued from the permutation structure and sustained public analysis, not merely from the number 1024.

16.3 Specific caveats

- The four CroackBoxes are affine-equivalent variants of one S-box, not four independent random permutations.
- No wide-trail lower bound is currently supplied for active S-boxes across rounds.
- No formal bound is supplied for ARX differential or linear probabilities.

- The constant generator is reproducible but intentionally noncryptographic; its role is transparency, not pseudorandomness against an adversary.
- Sixteen rounds are a design choice pending stronger reduced-round attacks and an explicit security margin.
- The table implementation is not constant-time with respect to message bytes.
- Tree mode has its own domain and must not be substituted silently for the sequential mode.

17 Experimental analysis

17.1 Current test categories

The repository tests:

- S-box bijectivity, inverse correctness, differential uniformity 4, and absence of fixed/opposite-fixed points;
- table/bitslice equivalence for all 256 byte values in every box;
- fixed digest vectors and incremental/one-shot equivalence;
- padding boundaries and arbitrary update segmentation;
- exact permutation inversion for every round count 1 through 16;
- scalar/AVX2 four-way equivalence;
- tree determinism across thread counts and sensitivity to a one-byte file change;
- reduced-round avalanche, rotational distance, sampled linear bias, differential bit bias, simple invariants, and deliberately truncated collision smoke tests.

17.2 Avalanche observations

The bundled CSV reports a 2048-bit state avalanche experiment. A one-round average of approximately 585 changed bits rises to approximately 1024 by round 2 and remains near half the state thereafter.

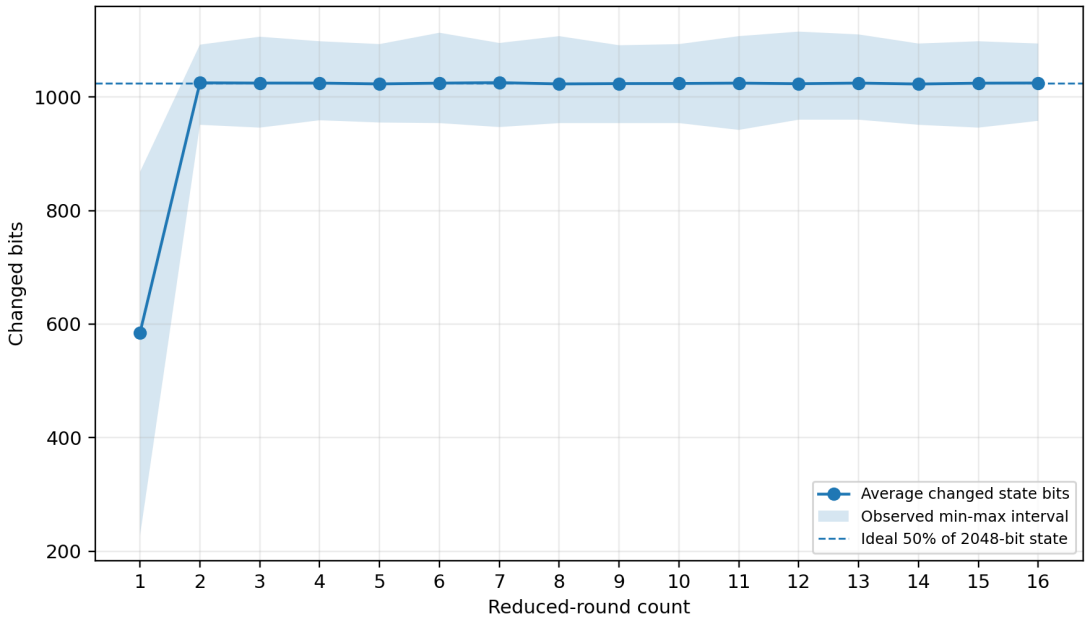
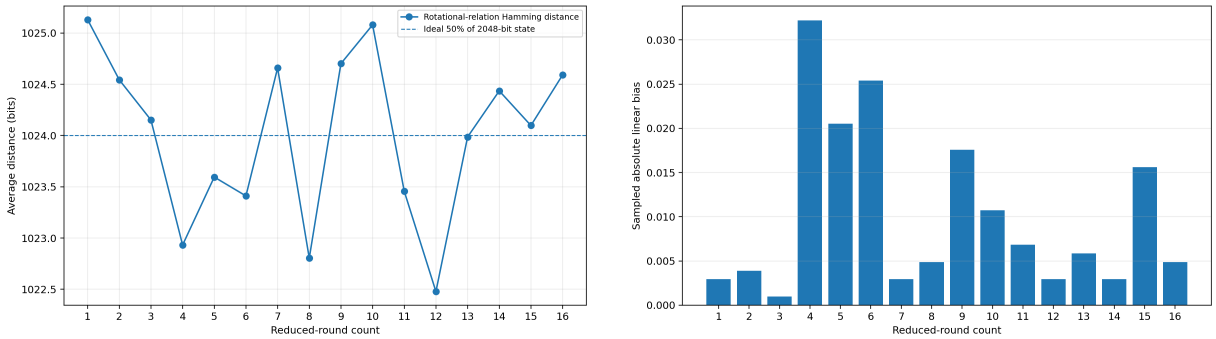


Figure 4: Observed state avalanche across reduced-round counts. The shaded band is the sampled minimum-to-maximum interval, not a confidence interval.

17.3 Rotational and sampled linear observations



(a) Rotational-relation distance.

(b) Sampled absolute linear bias.

Figure 5: Exploratory reduced-round measurements. Sample size and mask selection limit interpretation.

17.4 Analysis table

Table 6: Bundled reduced-round measurements.

Rounds	Avalanche avg	Min	Max	Rot. distance	Abs. bias
1	585.064	228	868	1025.131	0.002930
2	1024.833	951	1092	1024.544	0.003906
3	1024.437	946	1106	1024.152	0.000977
4	1024.418	959	1098	1022.932	0.032227
5	1022.891	955	1093	1023.595	0.020508
6	1024.318	954	1113	1023.411	0.025391
7	1025.199	947	1095	1024.659	0.002930
8	1022.883	954	1107	1022.804	0.004883
9	1023.484	954	1091	1024.702	0.017578
10	1023.688	954	1093	1025.080	0.010742
11	1024.365	942	1107	1023.456	0.006836
12	1023.258	960	1115	1022.478	0.002930
13	1024.434	960	1110	1023.986	0.005859
14	1022.685	951	1094	1024.436	0.002930
15	1024.256	946	1098	1024.098	0.015625
16	1024.557	958	1094	1024.591	0.004883

Security status

Passing avalanche, statistical, fuzzing, sanitizer, and inversion tests does not demonstrate collision, preimage, or second-preimage resistance. Such tests are valuable for implementation quality and for finding obvious structure, but they are not substitutes for cryptanalysis.

18 Recommended cryptanalytic program

Before any production claim, the following work is especially important:

1. derive lower bounds on active S-boxes across the lane and braid layers;
2. search differential and linear trails with MILP, SAT, SMT, or CP techniques;
3. study rotational-XOR, rotational-addition, and carry-dependent ARX effects;
4. test invariant subspaces, fixed points, short cycles, slide relations, and symmetries induced by parity alternation;
5. investigate rebound, boomerang, rectangle, integral, and impossible-differential distinguishers;
6. search for meet-in-the-middle and splice-and-cut decompositions of reduced rounds;
7. model algebraic degree growth and equation-system sparsity;
8. assess multicollision, herding, expandable-message, and tree-encoding behavior;
9. validate security margins by publishing the strongest attacks on reduced variants;
10. freeze a candidate version only after independent review and public challenge.

19 Performance and cache behavior

19.1 Operation profile

The core round uses byte substitution, 64-bit XOR, modular addition, fixed rotations, and compile-time lane movement. It contains no data-dependent branches in the permutation itself. The table profile has data-dependent loads from a 1 KiB S-box set; the bitsliced profile replaces those loads with Boolean operations.

19.2 Local benchmark snapshot

The development README records the following 64 MiB measurements:

Table 7: Development-container performance snapshot.

Profile	Approximate time
v0.2 sequential table	1.40 s
v0.3 sequential table	1.17 s
v0.3 scalar bitsliced	2.15 s
v0.3 tree, 8 threads + AVX2	0.11 s

These values are environment-specific and are not portable guarantees. The tree figure also measures a different domain and algorithmic mode from the sequential digest.

20 Command-line and API summary

20.1 CLI

Listing 4: Primary commands.

```
./frogbard -s "Libertas per Croack!"
./frogbard -f archive.tar
printf abc | ./frogbard -f -
./frogbard -T archive.tar
./frogbard -T archive.tar -j 16
./frogbard --backend
./frogbard --self-test
```

20.2 Streaming API

Listing 5: Sequential C API.

```
void frogbard_init(frogbard_ctx *ctx);
void frogbard_update(frogbard_ctx *ctx,
                     const void *data, size_t len);
void frogbard_final(frogbard_ctx *ctx,
                    uint8_t out[64]);
void frogbard_hash(const void *data, size_t len,
                   uint8_t out[64]);
```

The final function erases the temporary final block and context through a volatile byte loop after emitting the digest.

20.3 Research API

Listing 6: Reduced-round and inverse API.

```
void frogbard_permute_reduced(uint64_t state[4][8],
                              unsigned rounds);
void frogbard_permute_inverse_reduced(uint64_t state[4][8],
                                      unsigned rounds);
void frogbard_hash_reduced(const void *data, size_t len,
                           unsigned rounds, uint8_t out[64]);
```

Round counts above 16 are clamped. Reduced hash calls map zero rounds to one round.

21 Versioning and interoperability

Changing any phrase byte, generator detail, constant, S-box, rotation, lane index, operation order, padding byte, tree domain, or final marker changes the function. Such a change must create a new version identifier and new fixed vectors. Implementations should expose the version string `0.3-experimental` and must not label tree digests as sequential FrogBard digests.

For long-term interoperability, a future frozen release should define:

- an algorithm identifier and multihash-style code;
- canonical textual encoding and filename conventions;
- a stable endianness test corpus across architectures;
- a versioned tree-mode identifier;
- explicit deprecation behavior for experimental versions.

22 Conclusion

FrogBard-512 v0.3 is a concrete and reproducible research construction: a 2048-bit four-voice permutation, four public affine-equivalent CroackBoxes, ARX voice mixing, a cross-voice braid, explicit lane permutations, a 1024-bit-rate sponge-like sequential mode, and a separately domain-separated tree mode. Its implementation has substantial consistency, inversion, differential-smoke, fuzzing, and backend-equivalence tests. Those engineering results make the design suitable for further research. They do not yet justify deployment as a trusted cryptographic standard.

A Compact round pseudocode

Listing 7: Normative high-level round skeleton.

```
function Round(V, r):
    for voice v = 0..3:
        for lane i = 0..7:
            V[v][i] XOR= RC[r][v][i]

        apply CroackBox[(v+r) mod 4] to every byte of V[v]
        MixVoice(V[v], v, r)

    CrossVoiceBraid(V, r)   // exact sequential update order
    PermuteLanes(V, r)
```

B Constant-generation pseudocode

Listing 8: CroackBox derivation.

```
for phrase index j in 0..3:
    counter = 0
    repeat:
        state = Mix64("FrogBard-512/SBOX/v1", phrase[j], counter)
        pin   = FisherYatesPermutation(state, SplitMix64)
        pout  = FisherYatesPermutation(state, SplitMix64)
        inxor  = low_byte(next SplitMix64 output)
        outxor = low_byte(next SplitMix64 output)

        for x in 0..255:
            a = BitPermute(x XOR inxor, pin)
            b = AES_SBOX[a]
            S[x] = BitPermute(b, pout) XOR outxor

        counter += 1 if S has a fixed or opposite-fixed point
    until accepted
```

C Artifact checksums

Table 8: SHA-256 checksums of principal v0.3 artifacts used for this document.

Artifact	SHA-256
Source archive	2870859f42e6ec893ba3a53c53a9302edc2193e3240c41fbae20c3fb06445d43
`tools/gen_tables.py`	b67ad1e7d5260e71eb83002bef7d6234467d1850bb473bdb68b98c3ef6c04f17
`frogbard_tables.h`	7a4d1b7bdd9aad454ce7bb513d9126608695afea2db33b5ab26a900608dabd9d
`frogbard.c`	1aff8689129783e2c89bfd2dd79af11f5af5d6c7020b2414e26819f672362c7c

D License and third-party notice

The FrogBard source is distributed under the MIT License, copyright 2026 Víctor Duarte Melo. The bitsliced AES S-box circuit in `frogbard_aes_sbox.inc` is adapted from BearSSL code by Thomas Pornin, also distributed through PQClean, and is a software translation of the Boyar-Peralta Boolean circuit. The repository contains the complete applicable notice.

E Selected background references

1. National Institute of Standards and Technology, *FIPS PUB 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, 2015.
2. National Institute of Standards and Technology, *FIPS PUB 197: Advanced Encryption Standard (AES)*.
3. Guido Bertoni, Joan Daemen, Michael Peeters, and Gilles Van Assche, *Cryptographic Sponge Functions*.
4. Joan Boyar and Rene Peralta, work on compact and low-depth Boolean circuits for the AES S-box.
5. Daniel J. Bernstein, *The Salsa20 Family of Stream Ciphers*, for ARX design background.

Libertas per Croack!

Part II

Empirical Validation and Updated Analysis

Insane-profile run: 19 June 2026

Key finding

The evaluation is a broad engineering and statistical campaign. It can reveal implementation defects, obvious non-random structure, backend divergence, and runtime safety issues. It cannot substitute for independent cryptanalysis of the 2048-bit permutation.

1 Evaluation objectives and evidence model

The campaign was designed to test five distinct claims:

1. **Conformance:** all supported builds and backends produce identical fixed and streaming digests.
2. **Implementation safety:** memory, undefined behavior, leaks, and concurrency defects are not detected by multiple independent tools.
3. **Operational robustness:** files, pipes, Unicode paths, embedded NUL bytes, thread counts, read-only execution, limited memory, and adversarial update segmentation behave consistently.
4. **Statistical smoke quality:** large deterministic output streams do not show anomalies under general-purpose randomness batteries.
5. **Reduced-round behavior:** diffusion, sampled linear/differential bias, simple invariants, truncated-collision behavior, and exact inversion are measured across reduced variants.

These are evidence categories, not proofs. In particular, passing statistical tests does not imply collision resistance, and a million avalanche samples do not establish a lower bound on differential probability.

1.1 Generated streams

The counter stream hashes 64-byte messages whose first two 64-bit words encode a counter i and $i \cdot$ `9e3779b97f4a7c15` in little-endian form. Fixed phrase material is embedded in the same message. The differential stream emits

$$H(M_i) \oplus H(M_i \oplus e_{i \bmod 512}),$$

where the flipped input bit cycles through the 512-bit message. This second stream directly probes the distribution of one-bit output differences.

1.2 Artifact scale

The report directory occupied approximately 17 GiB. Two raw files dominated storage: an 8 GiB counter stream and an 8 GiB differential stream. Remaining binaries, logs, coverage data, AFL state, and tools occupied only tens of megabytes. This exposed a harness documentation defect: the profile message claimed 4 GiB of stored data while the implementation stored 8 GiB per stream.

2 Execution environment

3 Result accounting and harness corrections

The raw report records 250 passes, nine failures, and six warnings. The labels must be interpreted from exit status and output content rather than counted mechanically.

Interpretation note

The Cppcheck warning should not be dismissed solely as a false positive. The current control flow passes `NULL` for the right child when `child_count == 1`; the dereference is guarded by the child count in the implementation, but a clearer API contract or explicit branch would improve analyzability and defensive safety.

Operating system	Ubuntu 26.04 LTS (Resolute Raccoon), Linux 7.0.0-22-generic
Processor	AMD Ryzen 9 5950X, 16 cores / 32 threads, AVX2, AES, PCLMULQDQ, SHA-NI
Cache	512 KiB aggregate L1d, 8 MiB aggregate L2, 64 MiB L3
Memory	121 GiB reported physical memory, 68 GiB swap configured, swap unused at capture
Compiler/linker	Clang 21.1.8 and LLD 21.1.8; GCC 15.2.0 also present
Analysis tools	Valgrind 3.26.0, Cppcheck 2.19.0, clang-tidy 21.1.8, perf 7.0.0
Fuzz/statistical tools	libFuzzer, AFL++ 4.33c, PractRand 0.96, Dieharder 3.31.1, ENT
Parallel jobs	32

Table 2: Environment captured by the harness before the run.

Record class	Count	Interpretation
Raw PASS	250	Completed as expected with exit status 0.
PractRand raw FAIL	2	PractRand reached exactly 8 GiB and reported "no anomalies in 270 test result(s)." Exit 141 came from SIGPIPE in the upstream infinite generator after the consumer closed.
Interrupted raw FAIL	6	Hyperfine, GNU time, perf, and one small-stack test ended with exit 130 (SIGINT); no cryptographic failure was observed.
Cppcheck raw FAIL	1	One possible null-pointer dereference in `frogbard_tree.c`; the report also contains extensive third-party PractRand findings because the scan scope was too broad.
WARN	6	The warning lines correspond to the six interrupted tests rather than six additional failures.

Table 3: Corrected interpretation of the harness summary.

4 Build, conformance, and reproducibility

The following build families completed: native optimized, portable, strict warnings, constant-time bitsliced, hardened, debug, UBSan, ASan, static library, portable library, analysis tool, and fuzz-smoke. Scalar builds at `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, and `-Oz` passed self-tests and digest-equivalence checks.

Conformance checks covered:

- fixed sequential and tree vectors;
- one-shot versus arbitrarily segmented streaming updates;
- table CroackBoxes versus the bitsliced Boyar–Peralta path for all 256 byte values in all four boxes;
- scalar versus AVX2 four-way hashing;
- native, portable, strict, constant-time, and hardened digest identity;
- tree determinism across thread counts;
- Unicode filenames, embedded NUL bytes, standard input, and FIFO input;
- all relevant padding boundaries and a single-byte file mutation;
- 128 concurrent independent process invocations producing one unique digest;

- generated tables reproduced byte-for-byte from the four public phrases;
- two clean builds producing bit-for-bit identical binaries.

Binary inspection confirmed PIE, RELRO, `BIND_NOW`, a non-executable stack, stack-protector linkage, no unexpected RPATH, and AVX2 opcodes in the vector backend.

5 Memory safety, undefined behavior, and concurrency

Tool	Outcome
AddressSanitizer + LeakSanitizer	Self-test completed; no sanitizer report.
UndefinedBehaviorSanitizer	Instrumented build and self-test completed.
ThreadSanitizer	Tree-mode run completed without a reported race.
Valgrind Memcheck	Self-test and streaming-file runs: zero errors; all heap blocks freed.
Valgrind Helgrind	Tree mode: zero errors.
Valgrind DRD	Tree mode: zero errors.
Clang Static Analyzer	Completed without reported core defects.
clang-tidy	Core analysis completed.

Table 4: Dynamic and static implementation-safety evidence.

These results are especially relevant to tree mode because it uses atomic work claiming, `pread()`, worker threads, and four-message batches. The absence of detected races across TSan, Helgrind, and DRD is stronger than reliance on one tool, although it still cannot prove the absence of all concurrency defects.

6 Dynamic testing and fuzzing

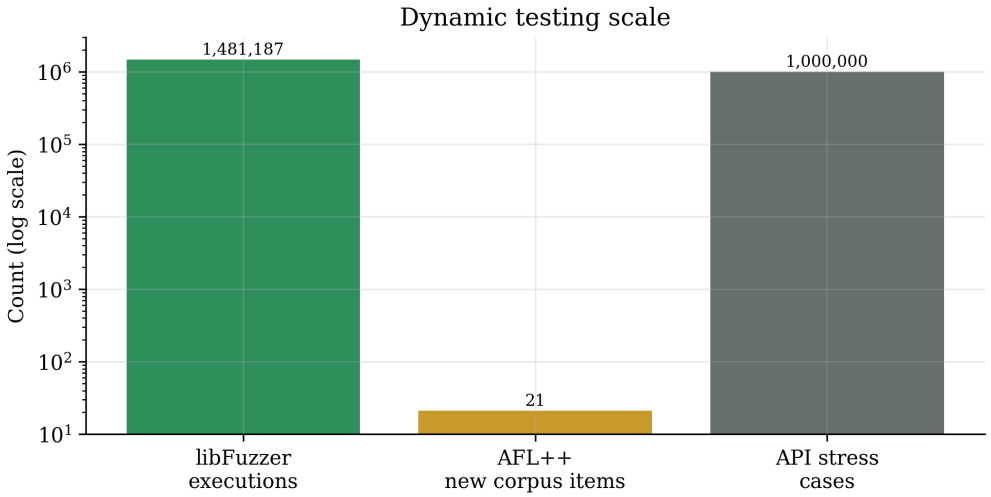


Figure 2: Scale of the principal dynamic tests. The logarithmic axis is used because API and libFuzzer counts are much larger than the number of AFL++ corpus additions.

6.1 Randomized API stress

One million cases compared one-shot and randomly segmented streaming evaluation. Every case also tested four-way equal-length hashing against four scalar calls. For non-empty messages, a uniformly selected input bit was flipped and the output Hamming distance was recorded. No mismatch was reported.

6.2 libFuzzer

The one-hour libFuzzer run completed 1,481,187 executions with zero crashes. The final state reported coverage value 527, feature count 1541, 51 corpus entries totaling 243 KiB, approximately 411 executions per second, and 432 MiB RSS.

6.3 AFL++

AFL++ ran for one hour, found 21 new corpus items, reported 63.27% coverage in its own instrumentation view, and saved zero crashes and zero timeouts.

7 Full-hash avalanche

For a 512-bit idealized output, the Hamming distance after a random one-bit input change follows approximately Binomial$(512, 1/2)$, with

$$\mathbb{E}[D] = 256, \qquad \sigma_D = \sqrt{512 \cdot \tfrac{1}{2} \cdot \tfrac{1}{2}} = \sqrt{128} \approx 11.313708.$$

Across one million randomized non-empty messages, the observed statistics were

$$\bar{D} = 256.002533, \quad \sigma = 11.310838, \quad D_{\min} = 201, \quad D_{\max} = 314.$$

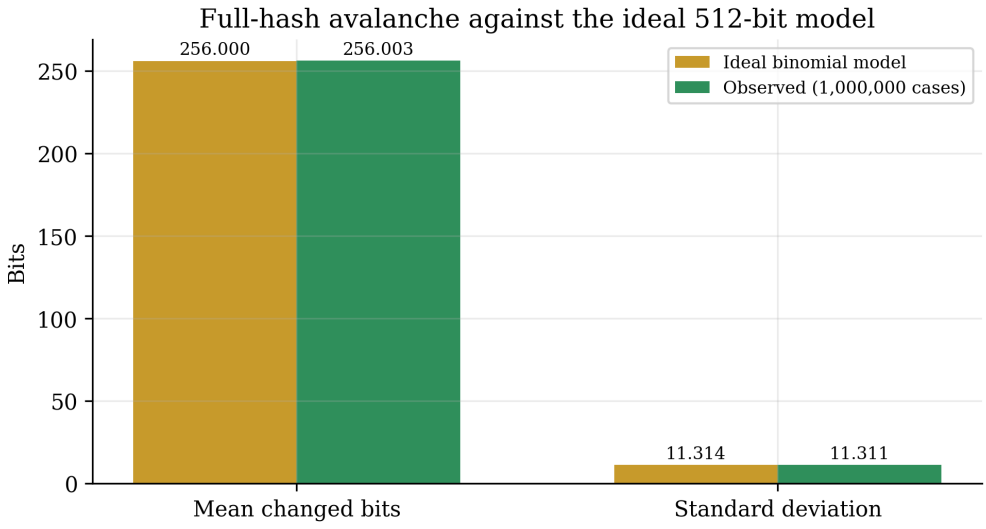


Figure 3: Observed full-hash avalanche mean and standard deviation compared with the ideal 512-bit binomial model. Agreement at this scale is a useful sanity check, not a security proof.

The difference between observed and ideal mean is approximately 0.002533 bits, and the observed standard deviation differs from the ideal value by approximately 0.00287 bits. This is unusually close, but it only describes the sampled distribution under the harness input generator.

8 Reduced-round permutation behavior

8.1 State avalanche

The 2048-bit internal permutation changes an average of 588.003 bits after one round. After two rounds, the mean reaches 1023.946 bits, effectively half the state, and remains near 1024 through round 16.

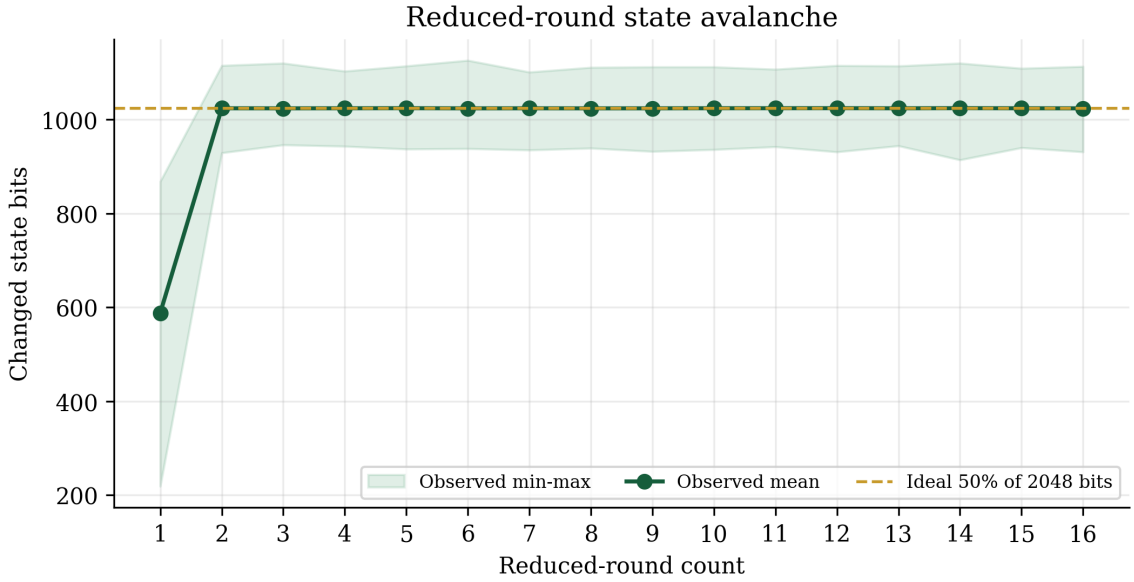


Figure 4: Reduced-round state avalanche. The shaded region is the observed sample minimum-to-maximum interval, not a confidence interval.

The one-round minimum of 219 and maximum of 868 demonstrate incomplete first-round diffusion, which is expected because each input bit has traversed only one local substitution/mixing/braid/lane schedule. The sharp transition by round two is encouraging for diffusion, but active-S-box lower bounds remain unproved.

8.2 Sampled differential and linear biases

Each differential scan used 65,536 samples and 1,024 selected input differences. Each linear scan used 65,536 samples and 2,048 selected masks. These scans are exploratory and sparse relative to the full 2^{2048} state space.

8.3 Inverse and simple invariants

The inverse stress test verified exact forward/inverse recovery for 16,384 random samples at every prefix length from one through 16 rounds. Four simple structured input patterns were checked at rounds 1, 2, 3, 4, 6, 8, 12, and 16; none preserved equal voices or equal lanes.

8.4 Truncated-collision smoke tests

Observed attempts ranged from 69,279 to 141,542. All values are plausible for a random 32-bit birthday experiment. No full-width collision search was attempted or implied.

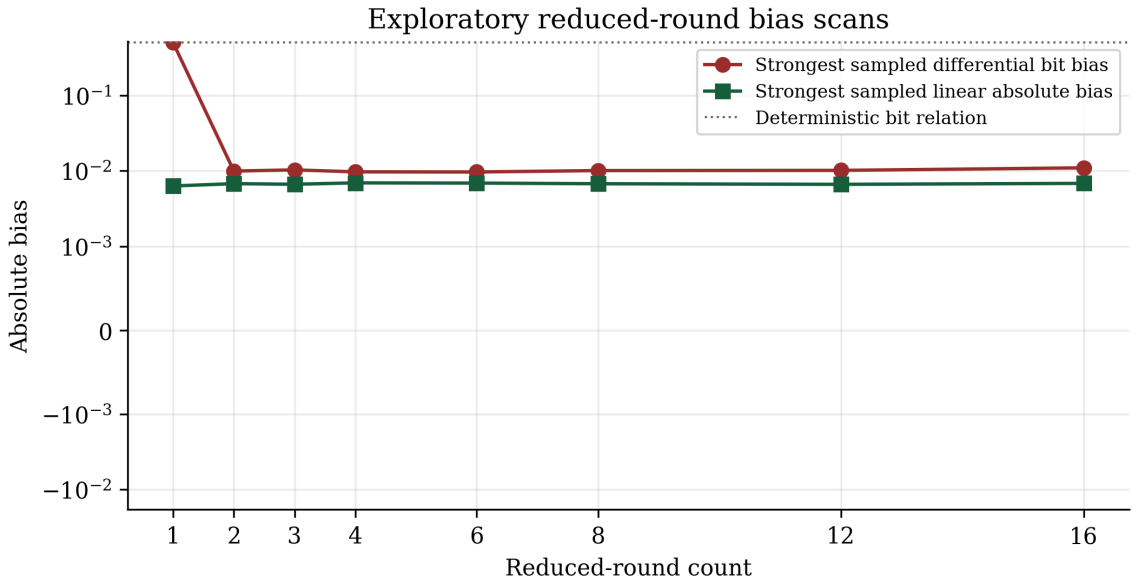


Figure 5: Strongest sampled differential bit bias and sampled linear absolute bias. The one-round differential value 0.5 indicates a deterministic relation in the restricted scan; from two rounds onward the strongest sampled values are around 10^{-2}.

9 Large-stream statistical testing

9.1 PractRand

Both the counter stream and differential stream were consumed to exactly 2^{33} bytes (8 GiB). PractRand 0.96 reported 270 core test results per stream and no anomalies. Runtime was 837 seconds for the counter stream and 1,611 seconds for the differential stream.

The harness classified both runs as exit 141 failures because the infinite producer attempted one more write after PractRand closed the pipe at the requested length. The statistical result files themselves are successful.

9.2 Dieharder

Across both streams, 224 assessments passed, four were marked weak, and none failed. The weak results were isolated rather than repeated across both streams in the same obvious pattern.

9.3 ENT and internal byte statistics

Metric	Counter stream	Differential stream
Bytes	8,589,934,592	8,589,934,592
Entropy (bits/byte)	7.999999981	7.999999979
Chi-square, 256 bins	229.598844	251.431989
ENT exceedance probability	87.17%	55.14%
Arithmetic byte mean	127.5013	127.5008
Serial correlation	-0.000018002	-0.000004567
Optimum compression	0%	0%

Table 5: Selected byte-level measurements for the two 8 GiB streams.

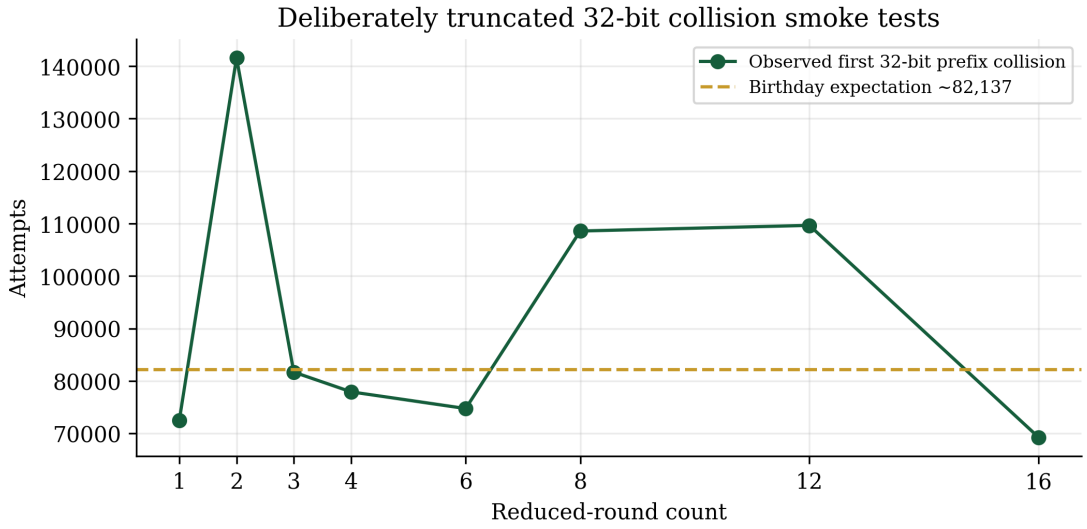


Figure 6: First observed collisions on an intentionally truncated 32-bit output prefix. The dashed line is the approximate birthday expectation $\sqrt{\pi/2}\,2^{16}$. These tests validate the search harness and coarse output behavior; they say nothing direct about 512-bit collision resistance.

10 Coverage and tested surface

File	Line coverage	Branch coverage	Function coverage
`frogbard.c`	92.53%	87.65%	97.50%
`frogbard_tree.c`	71.04%	48.25%	100.00%
`frogbard_cli.c`	57.35%	47.44%	80.00%
Total	**80.50%**	**66.10%**	**96.30%**

Table 6: LLVM coverage report.

The next coverage priority is not additional happy-path hashing. It is branch coverage of CLI failures, partial-I/O errors, allocation failures in tree mode, short reads, corrupted metadata, unsupported file types, thread-creation failures, and explicit unary-parent handling.

11 Runtime profile and incomplete performance evidence

Two one-hour fuzzing phases dominated guaranteed runtime. Statistical stream generation and testing added roughly 1.3 hours. The final benchmark phase was manually interrupted after Hyperfine had run for 768 seconds, so this campaign does not provide a clean new throughput comparison.

The earlier development snapshot remains informative but non-normative:

12 Known harness and implementation issues

12.1 Harness issues

1. **PractRand/SIGPIPE classification.** A bounded consumer closing an infinite producer is normal; the pipeline wrapper should explicitly accept producer exit 141 when PractRand exits successfully and its result file reaches the requested length.

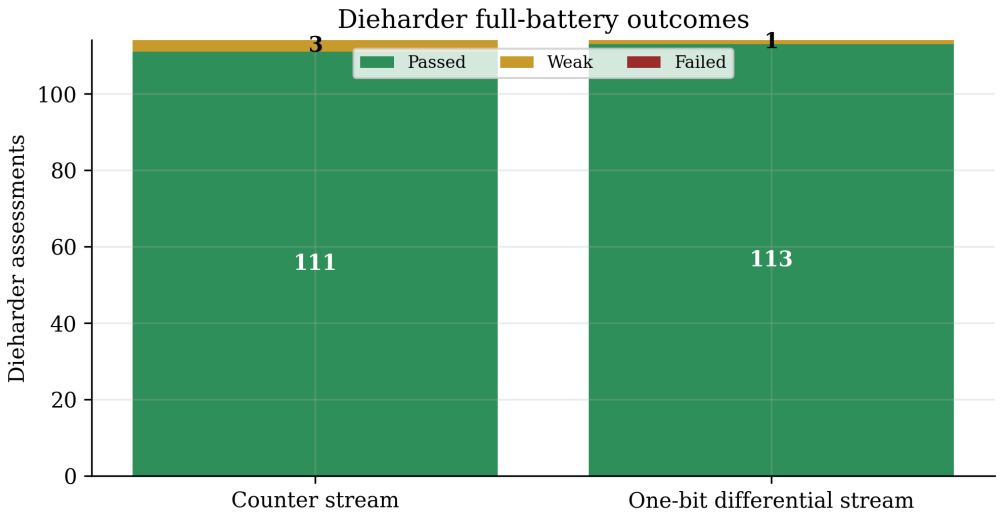


Figure 7: Dieharder assessments. Counter stream: 111 passed, 3 weak, 0 failed. Differential stream: 113 passed, 1 weak, 0 failed. Occasional weak p-values are expected in large batteries and require repetition before interpretation.

Profile	Approximate time for 64 MiB
v0.2 sequential table	1.40 s
v0.3 sequential table	1.17 s
v0.3 scalar bitsliced	2.15 s
v0.3 tree, 8 threads + AVX2	0.11 s

Table 7: Historical local benchmark snapshot retained from the normative specification. New benchmarking was incomplete in the insane run.

2. **Disk-size documentation.** The insane profile stored two 8 GiB streams, not a total of 4 GiB. The help text and preflight free-space estimate should be corrected.
3. **Cppcheck scope.** Third-party PractRand sources should be excluded. Core and vendor reports should be separated.
4. **Interruption accounting.** Exit 130 should be classified as *incomplete/interrupted*, not as a test failure.
5. **Performance isolation.** Hyperfine, GNU time, and perf should run as a resumable stage so that a long statistical campaign cannot lose benchmark completeness.

12.2 Implementation issue requiring review

Cppcheck reported a possible dereference of `right` in `make_parent_message`. The call site deliberately passes `NULL` for a unary parent and supplies `child_count == 1`; the implementation appears to condition the copy on the child count. A defensive rewrite should nevertheless make the condition explicit before any pointer expression and add a unit test that directly calls the unary-parent path under ASan/UBSan.

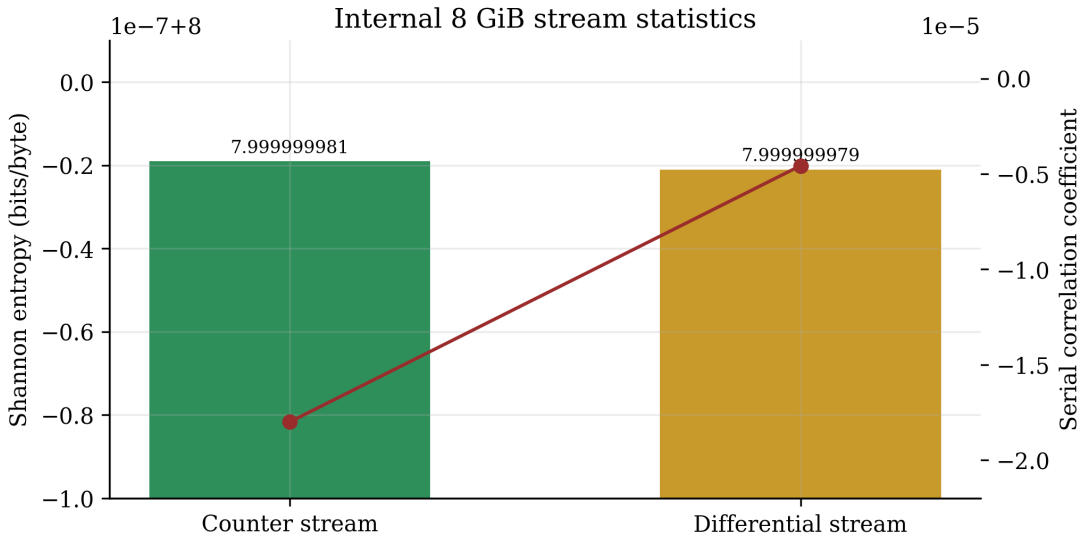


Figure 8: Entropy and serial correlation for the two stored 8 GiB streams. The vertical entropy scale is intentionally narrow to expose tiny deviations from 8 bits per byte.

13 What the evidence supports

Key finding

The results strongly support the claim that the v0.3 implementation is internally consistent, reproducible, portable across tested optimization profiles, free of detected memory/race defects, and capable of producing output streams without anomalies under the executed general-purpose statistical batteries.

The results do *not* support any of the following stronger claims:

- a proof or conservative bound for 256-bit collision resistance;
- a proof or conservative bound for 512-bit preimage or second-preimage resistance;
- indifferentiability from a random oracle;
- resistance to rebound, boomerang, rotational, algebraic, integral, meet-in-the-middle, invariant-subspace, or structural attacks;
- an adequate 16-round security margin;
- side-channel security for keyed use;
- production readiness or standardization suitability.

14 Recommended next research program

The strongest next step is not another terabyte of randomness tests. It is structured cryptanalysis:

1. formalize the round as bit-vector constraints and search reduced-round differentials with SAT/SMT/MILP;
2. derive lower bounds on active CroackBoxes across the voice mixer, braid, and lane permutation;
3. model modular-addition carries and rotational-XOR/addition relations;

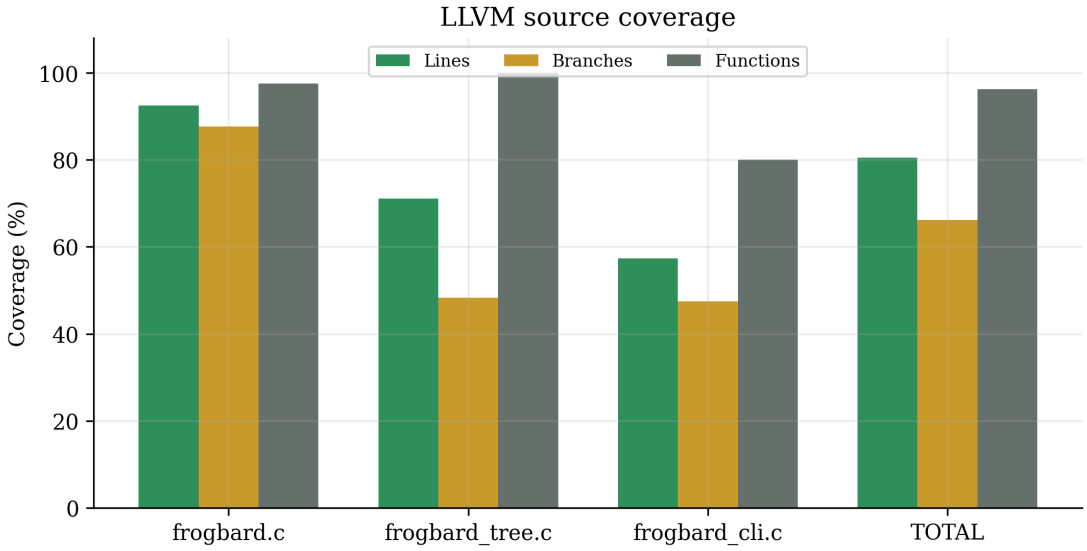


Figure 9: LLVM source coverage from the integrated coverage run. The cryptographic core is substantially better covered than CLI and tree error paths.

4. search invariant subspaces, weakly coupled lane sets, fixed points, short cycles, and parity-induced slide structure;
5. measure algebraic degree growth and equation sparsity through reduced rounds;
6. attempt rebound, boomerang, rectangle, impossible-differential, and integral distinguishers;
7. investigate meet-in-the-middle and splice-and-cut decompositions around alternating round parity;
8. extend the tree proof obligations: prefix-freeness, domain separation, unary-parent encoding, root binding, and multicollision/herding behavior;
9. repeat performance measurements with CPU affinity, fixed governor, warmup, confidence intervals, and exact bytes-per-cycle reporting;
10. publish reduced-round challenges and reward the strongest attacks before freezing a candidate version.

15 Conclusion

The new campaign substantially raises confidence in FrogBard-512 as a research implementation. It survived a broad matrix of compilers and optimization levels, multiple memory and concurrency analyzers, two independent one-hour fuzzers, a million randomized API cases, exact inverse tests across all round prefixes, 16 GiB of stored statistical streams, two 8 GiB PractRand runs, two full Dieharder batteries, reproducibility checks, hardening inspection, and extensive functional edge cases.

The most striking numerical result is the full-hash avalanche distribution: a mean of 256.002533 changed bits and standard deviation 11.310838 across one million one-bit perturbations, nearly identical to the ideal binomial model. The permutation reaches approximately half-state diffusion by two rounds. No completed general-purpose statistical test reported a failure.

At the same time, the campaign exposed useful engineering work: correct SIGPIPE accounting, accurate disk estimates, cleaner Cppcheck scope, explicit unary-parent pointer handling, improved CLI/tree branch coverage, and resumable performance stages. Most importantly, no amount of automated smoke testing removes the need for independent cryptanalysis. Version 0.3-experimental should therefore be viewed as a well-tested research candidate, not a trusted standard.

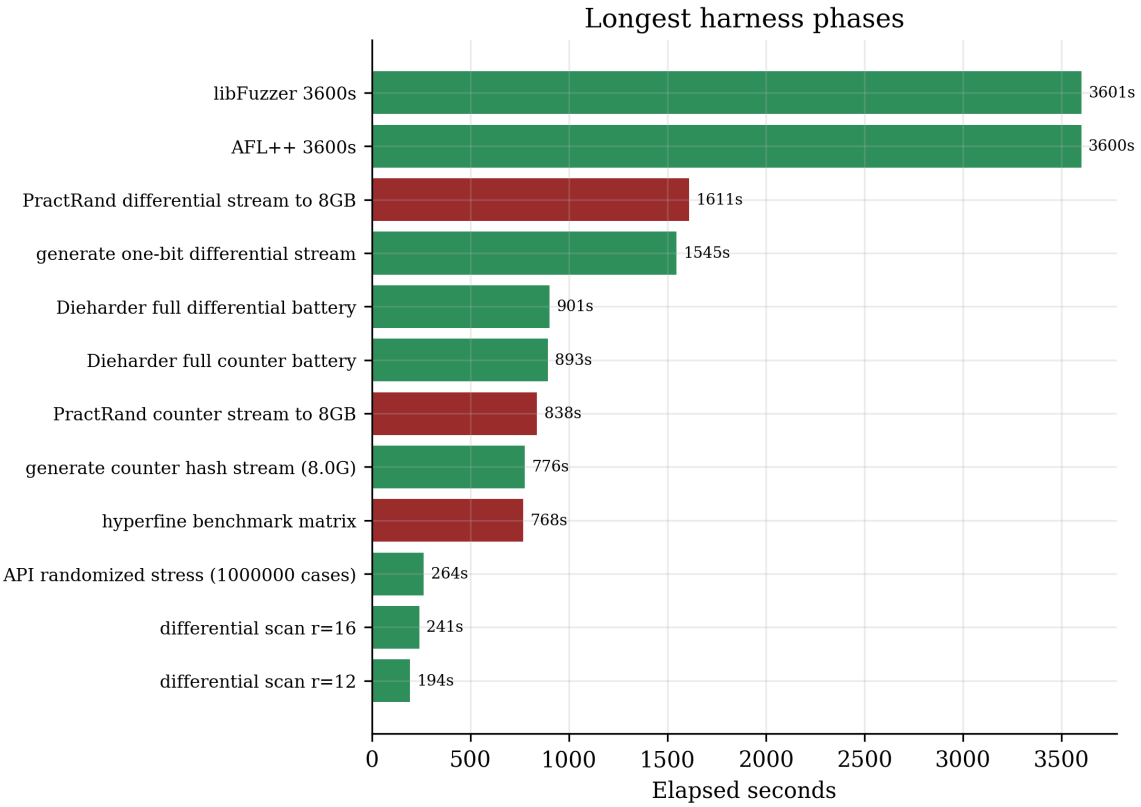


Figure 10: Longest phases recorded by the harness. Red bars are raw failure records; the two long PractRand bars completed statistically, while the Hyperfine phase was interrupted.

Libertas per Croack!

A Selected exact reduced-round measurements

Rounds	Avalanche avg	Min	Max	Rot. distance	Linear abs. bias
1	588.003	219	868	1024.083	0.007385
2	1023.946	929	1115	1024.199	0.002747
3	1023.851	946	1120	1024.273	0.002930
4	1023.969	943	1103	1023.623	0.005432
5	1024.128	937	1114	1024.274	0.006653
6	1023.831	938	1126	1023.898	0.003418
7	1023.909	935	1101	1024.132	0.001648
8	1023.821	939	1111	1024.229	0.004395
9	1023.851	932	1112	1024.106	0.003174
10	1024.059	936	1112	1023.929	0.000122
11	1024.086	942	1107	1023.878	0.000488
12	1024.169	931	1115	1023.916	0.001343
13	1023.948	944	1114	1024.118	0.003784
14	1024.198	914	1120	1024.088	0.001404
15	1023.956	940	1109	1023.977	0.003479
16	1023.845	931	1113	1024.075	0.000244

B Selected exact scan results

Rounds	Differential bias	Linear bias	32-bit collision attempts	Invariant smoke
1	0.50000000	0.00631714	72,464	no equality retained
2	0.00994873	0.00677490	141,542	no equality retained
3	0.01031494	0.00665283	81,686	no equality retained
4	0.00971985	0.00694275	77,927	no equality retained
6	0.00967407	0.00689697	74,749	no equality retained
8	0.01010132	0.00675964	108,565	no equality retained
12	0.01016235	0.00663757	109,640	no equality retained
16	0.01098633	0.00683594	69,279	no equality retained

Table 9: Selected exploratory reduced-round measurements.

C Raw summary excerpt

```
FrogBard-512 absurd test report
UTC finished: 2026-06-19T20:08:18+00:00
Profile: insane
Elapsed seconds: 16842
PASS: 250
FAIL: 9
SKIP: 0
WARN: 6

Important: implementation/statistical tests do not prove collision,
preimage, second-preimage, indifferentiability, or structural
cryptanalytic security.
```

D Evaluation artifact inventory

Artifact	Role
SUMMARY.txt	Raw harness summary and failed-test list.
results.tsv	Per-test status, return code, elapsed seconds, label, and log path.
ENVIRONMENT.txt	Kernel, OS, CPU, memory, filesystems, and tool versions.
stats/round-metrics.csv	Reduced-round avalanche, rotational distance, and sampled linear bias.
stats/practrand-*.txt	Final 8 GiB PractRand result summaries.
stats/dieharder-*.txt	Full Dieharder batteries for both generated streams.
internal-statistics.txt	Entropy, chi-square, monobit, serial correlation, and byte-count ranges.
coverage/report.txt	LLVM regions, functions, lines, and branches by source file.
logs/*.log	Commands, diagnostics, sanitizer output, fuzzer output, and tool transcripts.